TRIUNE BIBLICAL UNIVERSITY
GLOBAL EXTENSION USA

# Introduction to Artificial Intelligence

A Self-Contained Undergraduate Textbook

AI.101
Introduction to Artificial Intelligence
Year 1 \u2014 Semester 1

TRIUNE BIBLICAL UNIVERSITY GLOBAL EXTENSION USA

ISBN: 978-1-00000-101-1 (Assigned)
TRIUNE BIBLICAL UNIVERSITY GLOBAL EXTENSION USA © 2026

# Copyright and Publication Information

**Introduction to Artificial Intelligence**
TRIUNE BIBLICAL UNIVERSITY GLOBAL EXTENSION USA
AI.101

TRIUNE BIBLICAL UNIVERSITY GLOBAL EXTENSION USA © 2026. All rights reserved.
No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher.

Published by TRIUNE BIBLICAL UNIVERSITY GLOBAL EXTENSION USA.
ISBN: 978-1-00000-101-1 (Assigned)
This textbook is produced for the Bachelor of Artificial Intelligence and Robotics Engineering programme, course AI.101 Introduction to Artificial Intelligence.

Textbook Production Standard: each chapter is self-contained and includes learning objectives, prerequisite knowledge, core concepts, definitions, worked examples, mathematical derivations, algorithms and pseudocode, engineering applications, practical laboratory activities, programming exercises, case studies, review questions, MCQs, short-answer questions, numerical problems, programming/design exercises, a mini-project, and further reading.

*Version 1.0 — Final Edition*

# Contents


Foundations and History of Artificial Intelligence 6

Intelligent Agents and Environments 11

Problem Formulation and State-Space Search 16

Uninformed Search Algorithms 20

Informed Search Algorithms 25

Adversarial Search 30

Knowledge Representation and Logic 35

Reasoning Under Uncertainty and Bayesian AI 39

Expert Systems 43

Machine Learning and Neural Networks Overview 47

Natural Language Processing and Computer Vision 51

Robotics, Generative AI, Responsible AI, and the Future 55

Final Integrated AI Project: Autonomous Delivery Agent 59

Back Matter 60

Glossary . . . . . 60

Bibliography . . . . . 62
Subject Index . . . . . 63
Practical-Laboratory Index . . . . . 64
Recommended Software and Hardware Tools . . . . . 65

## Final Integrated Project

Autonomous Delivery Agent — end-of-course capstone

## Back Matter

Glossary
Bibliography
Subject Index
Practical-Laboratory Index
Recommended Software and Hardware Tools

CHAPTER 1

# Foundations and History of Artificial Intelligence

*What AI is, its milestones, and its place in engineering*

## Chapter Introduction

Artificial Intelligence (AI) is the branch of computer science and engineering concerned with building systems that perceive, reason, learn, and act in ways that would be called intelligent if performed by a human. For students of AI and Robotics Engineering, AI is not a mysterious black box but an engineered discipline grounded in mathematics, computation, and real-world application. This chapter traces how the field emerged, what "intelligence" means in an engineering context, and why AI matters to robots, autonomous vehicles, and intelligent machines. Engineers should leave this chapter able to look at any device - a thermostat, a robotic arm, a voice assistant - and say which characteristics of intelligence it exhibits, which it lacks, and whether labelling it "AI" is justified. The historical survey is included not as nostalgia but as a warning: each past boom was driven by a specific technical breakthrough (symbolic search, expert systems, backpropagation, deep networks) and each bust followed when promises outran capability. Understanding that cycle is part of being a responsible AI engineer.

## Learning Objectives

## Prerequisite Knowledge

Basic comfort with high-school mathematics (algebra, functions, elementary probability) and basic computer literacy. No prior programming or AI background is assumed; relevant concepts are introduced as needed.

## Core Concepts

**1. core_a.** What artificial intelligence is, and what it is not. AI studies how to make machines perform tasks that normally require human intelligence \u2014 recognizing speech, interpreting images, planning routes, diagnosing faults, playing games, and conversing. Crucially, AI is engineering: we specify objectives, build models, measure performance, and iterate.

**2. core_b.** The AI timeline: 1940s\u20131950s foundations (Turing, McCulloch & Pitts, von Neumann); the 1956 Dartmouth workshop that coined \u201cartificial intelligence\u201d; early symbolic AI (Logic Theorist, General Problem Solver); the first AI winter; the expert-systems boom of the 1980s; the second AI winter; the rise of machine learning, big data, and deep learning in the 2000s\u20132010s; and today\u2019s generative-AI era.

**3. core_c.** Intelligence as a multidimensional capacity: perception, memory, reasoning, learning, planning, communication, and adaptation. An engineered system need not be conscious; it must behave competently toward a goal.

**4. core_d.** Relationships among AI, machine learning (ML), deep learning (DL), and data science: AI is the broad field; ML is a data-driven subset of AI; DL is a multilayer-network subset of ML; data science is the practice of extracting knowledge from data, heavily overlapping with ML.

## Key Definitions & Terminology

****Artificial Intelligence (AI)**.**

The study and construction of agents that perceive their environment and take actions to maximize their chance of achieving goals.

****Intelligent agent**.**

A system that senses its environment through sensors and acts upon it through actuators or effectors.

****Rational agent**.**

An agent that acts to achieve the best expected outcome given its knowledge and objectives.

****Machine Learning (ML)**.**

AI systems that improve their performance on a task through experience with data, rather than through explicit programming of every rule.

****Deep Learning (DL)**.**

ML based on artificial neural networks with many layers, capable of learning hierarchical representations from raw data.

****Data Science**.**

The interdisciplinary practice of collecting, cleaning, analyzing, and interpreting data to support decisions.

## Worked Examples

**1. example_1.** A thermostat maintains room temperature by sensing the current temperature and switching heating on or off. It is a simple goal-seeking system, but it

has no memory, no learning, and no model of the world \u2014 it is not considered AI. Contrast this with a robot vacuum that builds a map of your home, learns which rooms get dirtiest, and plans efficient cleaning routes. The robot vacuum exhibits perception, memory, planning, and adaptation \u2014 hallmarks of AI.

### Algorithms & Pseudocode

- No formal algorithm in this chapter; instead, the \u201csense-think-act\u201d cycle is introduced as the organizing model for every intelligent system studied later in the book.

### Engineering Applications

Robotic manufacturing arms that adapt to part variation.

Autonomous vehicles that perceive traffic and plan safe routes.

Recommendation and diagnostic systems in healthcare and agriculture.

Voice assistants and chatbots that understand and generate language.

### Practical Laboratory Activity

- Demonstration: observe a simple reactive robot (or simulator) following a light source using two light sensors and two motors (Braitenberg-style vehicle). Discuss: is it \u201cintelligent\u201d? Which characteristics of intelligence does it show or lack?
- Class exercise: list five everyday devices and classify each as \u201cnon-intelligent,\u201d \u201csmart but rule-based,\u201d or \u201cAI,\u201d justifying each with the intelligence characteristics from this chapter.

### Case Study

**1. case_a.** Case: Autonomous warehouse robot (Amazon-style). A fleet of mobile robots transports shelves to picking stations. Each robot perceives its location (wheel encoders, camera, LiDAR), reasons about the shortest safe path, avoids other robots and people, and reports task completion. Perception \u2192 reasoning \u2192 action loop operates continuously. Engineering meaning: intelligence here is measured by throughput, safety, and energy use, not by conversation.

### Common Mistakes & Troubleshooting

- Confusing AI with automation: not every automated system is AI. Automation follows fixed rules; AI adapts and learns from data.

- Assuming intelligence requires consciousness or human-like thought. Engineering AI measures goal-directed behavior, not subjective experience.
- Overestimating current AI: today\u2019s systems are narrow \u2014 strong at one task, brittle outside it.

## Chapter Summary

This chapter established AI as an engineered discipline, surveyed its history from the Dartmouth workshop to generative AI, defined intelligence in behavioral terms, and clarified how AI relates to ML, DL, and data science. You should now be able to discuss what AI can and cannot do and why it is foundational to robotics engineering. Return to these definitions whenever a later chapter claims a system is "intelligent": ask what it perceives, what objective it optimises, and how its performance is measured.

## Key Terms

Artificial intelligence; intelligent agent; rational agent; sense-think-act cycle; machine learning; deep learning; data science; AI winter; Dartmouth workshop; narrow AI; general AI.

## Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** Which event is widely regarded as the birth of AI as a field?
A. The invention of the first digital computer
B. The 1956 Dartmouth workshop
C. The publication of Turing\u2019s 1950 paper
D. The first expert system (DENDRAL)

**Answer:** B

**q2.** A thermostat that switches heat on below 18\u00b0C and off above 21\u00b0C is best described as:
A. A rational learning agent
B. A reactive control system, not AI
C. A deep-learning system
D. A general AI

**Answer:** B

**q3.** Machine learning is best described as:
A. A superset of artificial intelligence
B. AI systems that improve through experience with data

C. Only symbolic rule-based systems
D. Identical to data science

**Answer:** B

### Short-Answer Questions

**1. s1.** Why is consciousness not required for a system to be classified as AI in engineering terms?

**2. s2.** Give one example of a task that early symbolic AI (1960s) handled well and one it handled poorly.

### Numerical / Problem-Solving Questions

**1. n1.** If a rule-based classifier correctly labels 86 out of 100 email messages, what is its accuracy as a percentage?

### Programming / Design Exercises

**1. p1.** Optional warm-up: install Python and write a program that prints the AI timeline milestones from this chapter as a numbered list. Confirm your environment runs before Chapter 5, where you will implement search algorithms.

### Mini-Project

Intelligence audit. Working in pairs, select one AI-powered product (for example, a voice assistant, a facial-unlock phone, or a robotic vacuum). Research its capabilities and limits, then prepare a two-page report identifying which intelligence characteristics it demonstrates, what data it uses, and where it might fail.

### Further Reading & References

Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th ed. Pearson. (Ch. 1)

Nilsson, N. J. (2010). The Quest for Artificial Intelligence. Cambridge University Press.

Turing, A. M. (1950). \u201cComputing Machinery and Intelligence.\u201d Mind, 59(236), 433\u2013460.

McCarthy, J. et al. (1955). \u201cA Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.\u201d

CHAPTER 2

# Intelligent Agents and Environments

*PEAS, rationality, and agent architectures*

## Chapter Introduction

Every AI system is, at heart, an agent: something that perceives and acts. This chapter formalizes the agent concept, introduces the rational-agent principle, classifies environments, and presents the agent architectures that will be used throughout the book. You will learn to describe any robot or AI program in terms of its percepts, actions, goals, and performance measure. The PEAS template is intended to be filled out for every robot or autonomous system you design, not memorised as an abstract table. A good performance measure is observable, measurable, and aligned with the task; a bad one rewards the wrong behaviour. Rationality is deliberately local: it concerns the best decision given the information actually available, not hindsight. This chapter's agent architectures form a complexity ladder - choose the simplest rung that satisfies the requirement, because every added rung costs development time, compute, and predictability.

## Learning Objectives

## Prerequisite Knowledge

Chapter 1 (what AI is). Basic set notation and functions are helpful but are introduced inline.

## Core Concepts

**1. core_a.** The agent: any entity that perceives its environment through sensors and acts upon it through actuators. A robot agent uses cameras, LiDAR, and encoders as sensors, and motors and grippers as actuators. A software agent uses keystrokes, network packets, or API calls.

**2. core_b.** PEAS description: for any agent, specify its Performance measure, Environment, Actuators, and Sensors. Example: an autonomous taxi. Performance = safe, fast, legal, comfortable trips; profit maximized. Environment = roads, other traffic, pedestrians, weather. Actuators = steering, accelerator, brake, horn, display. Sensors = cameras, radar, LiDAR, GPS, IMU, speedometer.

**3. core_c.** Rationality: at each time step, a rational agent selects the action expected to maximize its performance measure, given the percept sequence to date and its built-in knowledge. Rationality is not omniscience; it is the best decision given available information.

**4. core_d.** Agent architectures: (1) simple reflex \u2014 condition-action rules; (2) model-based reflex \u2014 maintains an internal model of the world; (3) goal-based \u2014 acts to achieve explicit goals; (4) utility-based \u2014 acts to maximize a utility function when goals conflict or are uncertain; (5) learning agent \u2014 improves over time through a performance element, critic, learning element, and problem generator.

## Key Definitions & Terminology

****Agent**.**

An entity that perceives and acts.

****Percept**.**

The complete sensory input at a given instant.

****Percept sequence**.**

The complete history of everything an agent has ever perceived.

****Actuator / Effector**.**

A means by which an agent acts on its environment.

****Rational agent**.**

An agent that acts to maximize expected performance given its percept sequence.

****Autonomy**.**

The extent to which an agent\u2019s behavior is determined by its own experience rather than built-in knowledge.

## Worked Examples

**1. example_1.** PEAS for a robotic vacuum cleaner: Performance = floor cleaned, time, energy, furniture avoided; Environment = rooms, obstacles, stairs, dust; Actuators = wheels, vacuum motor, brush, display; Sensors = bump sensors, infrared cliff detectors, wheel encoders, battery level.

## Algorithms & Pseudocode

- Table-driven agent: looks up the percept sequence in a table to select an action (impractical for long sequences).
- Simple reflex agent: if condition then action (condition-action rules).

## Engineering Applications

Industrial robot arm (welding): predefined reflex actions to sensor input.

Mars rover: model-based reflex with an internal world model and goal-based planning.

Autonomous drone delivery: utility-based routing balancing speed, battery, and no-fly zones.

## Practical Laboratory Activity

- Build PEAS tables for four agents: (a) autonomous car, (b) medical-diagnosis AI, (c) warehouse robot, (d) email spam filter.
- Simulator task: program a simple-reflex vacuum agent in a two-room grid world using condition-action rules (if dirty then suck; if right then move left, etc.). Measure rooms cleaned per minute.

## Programming Exercise

**Listing 2.1: Simple reflex agent in Python.**

```
# Simple-reflex vacuum agent (Python)
def reflex_agent(percept):
room, status = percept
if status == 'Dirty':
return 'Suck'
if room == 'A':
return 'Right'
return 'Left'

# Run on a short percept sequence
env = [('A','Dirty'), ('A','Clean'), ('B','Dirty'), ('B','Clean')]
for p in env:
print(p, '->', reflex_agent(p))
```

## Case Study

**1. case_a.** Case: Amazon warehouse drive unit. The robot\u2019s performance measure is shelves delivered per hour with zero collisions. Its environment is a grid of lanes shared with other robots and humans. It uses a model-based architecture: it maintains a map (internal model) and replans when blocked. Its autonomy is high \u2014 routes are computed onboard, though a central scheduler assigns tasks.

## Common Mistakes & Troubleshooting

- Treating rationality as perfect foresight. A rational agent can fail if its percepts are misleading; rationality concerns the decision process, not the outcome.

- Confusing actuators with sensors. Sensors bring information in; actuators send action out.
- Assuming all environments are fully observable. Most robotics environments are partially observable.

## Chapter Summary

Agents are the unifying model of AI. You can now describe any intelligent system with PEAS, judge rationality, classify environments, and choose an appropriate agent architecture. These concepts underpin the search, planning, and robotics chapters that follow. Use PEAS as a design checklist before implementing any agent in this book or in your projects.

## Key Terms

Agent; percept; actuator; PEAS; rational agent; rationality; simple reflex; model-based reflex; goal-based; utility-based; learning agent; autonomy; environment classification.

## Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** In PEAS, the \u201cP\u201d stands for:
A. Percept
B. Performance
C. Processor
D. Planning

**Answer:** B

**q2.** Which environment is partially observable?
A. Tic-tac-toe (full board visible)
B. Robot vacuum cleaning an unseen room
C. A calculator
D. A printed crossword puzzle

**Answer:** B

**q3.** A learning agent differs from other agents because it:
A. Has more sensors
B. Improves performance through experience
C. Is always rational
D. Needs no performance measure

**Answer:** B

### Short-Answer Questions

**1. s1.** Why are lookup-table agents impractical for most real tasks?

**2. s2.** Give one robotics example of a dynamic environment.

### Numerical / Problem-Solving Questions

**1. n1.** A delivery robot completes 48 successful deliveries and 2 collisions in 50 trips. If performance = successful trips / total trips, what is its performance measure?

### Programming / Design Exercises

**1. p1.** Extend Listing 2.1 so the agent remembers which rooms it has already cleaned and avoids re-entering a clean room unless necessary.

### Mini-Project

Agent taxonomy poster. In groups, pick six real AI/robot products and place each on a two-axis chart: (x) degree of autonomy, (y) architecture complexity from reflex to learning. Annotate each with its PEAS summary.

### Further Reading & References

Russell & Norvig (2020), Ch. 2 \u2014 Intelligent Agents.

Wooldridge, M. (2009). An Introduction to MultiAgent Systems. Wiley.

Braitenberg, V. (1984). Vehicles: Experiments in Synthetic Psychology. MIT Press.

CHAPTER 3

# Problem Formulation and State-Space Search

*Turning tasks into search problems*

## Chapter Introduction

Most AI problems can be cast as searching through a space of possible states to find a sequence of actions that reaches a goal. This chapter teaches problem formulation \u2014 the engineering act of converting a messy real-world task into a precise search problem \u2014 and introduces state-space representation, the foundation for every search algorithm in Chapters 4\u20136. Formulation is where most AI projects succeed or fail, yet it receives less attention than search algorithms. A well-chosen state abstraction makes a hard problem easy; a poor one makes an easy problem impossible. The worked examples in this chapter are therefore deliberately engineering-flavoured: route-finding for a mobile robot, tile rearrangement for a manipulator, and item-location for a warehouse picker. Practise formulating before you practise searching.

## Learning Objectives

## Prerequisite Knowledge

Chapter 2 (agents). Basic graph terminology (nodes, edges) is introduced here.

## Core Concepts

**1. core_a.** Problem formulation: a problem is defined by five components: (1) initial state; (2) actions available in each state; (3) transition model (result(state, action) \u2192 next state); (4) goal test; (5) path-cost function. A solution is a sequence of actions from the initial state to a goal state; an optimal solution minimizes path cost.

**2. core_b.** State space: the set of all states reachable from the initial state. A state-space representation is a graph whose vertices are states and whose edges are actions. For an n-node graph, the state space can be enormous \u2014 which is why search strategy matters.

**3. core_c.** Search tree vs. state space: the search tree is generated during search and may contain the same state multiple times reached by different paths; the state space

is the underlying graph. We search the tree but care about states.

**4. core_d.** Problem types: single-state (fully observable, deterministic), multiple-state (no sensors), contingency (partially observable or stochastic), and exploration (unknown environment). Most robotics problems are contingency problems.

## Key Definitions & Terminology

****State**.**

A configuration of the world relevant to the problem.

****State space**.**

All states reachable from the initial state.

****Node**.**

A data structure in the search tree (state, parent, action, path cost, depth).

****Solution**.**

An action sequence from initial state to a goal state.

****Optimal solution**.**

A solution with the lowest path cost.

****Path cost**.**

The numeric cost of a path, usually the sum of step costs.

## Worked Examples

**1. example_1.** Romania route-finding: states are cities; actions are driving between connected cities; transition model is the road map; goal test = arrive in Bucharest; path cost = distance in km. This classic example illustrates state space and search and maps directly to robot navigation (Chapter 12 and AI.307).

## Algorithms & Pseudocode

- Problem formulation recipe: identify states \u2192 actions \u2192 transition \u2192 goal test \u2192 cost.

## Engineering Applications

Robot arm motion planning (configurations are states).

Autonomous vehicle route planning (intersections are states).

Puzzle solving (Rubik\u2019s cube, 8-puzzle) as canonical AI examples.

## Practical Laboratory Activity

- On paper, draw the state-space graph for the 8-puzzle starting from a given tile arrangement; list neighbors of the start state.
- Formulate a warehouse-picking robot task (pick item, deliver to station) as a search problem: define states, actions, goal, cost.

## Case Study

**1. case_a.** Case: Autonomous delivery robot in a building. States = (room, battery-level, carrying-item?). Actions = move-to-adjacent-room, pick-up, drop-off, recharge. Transition model includes battery drain and door availability. Goal = item delivered to recipient. Path cost = time + energy. This formulation drives the planners in AI.311 and AI.407.

## Common Mistakes & Troubleshooting

- Choosing a state representation that is too detailed (huge state space) or too coarse (loses needed information).
- Forgetting the path-cost function, making it impossible to prefer shorter or cheaper solutions.
- Confusing the state space (fixed graph) with the search tree (grown during search).

## Chapter Summary

Problem formulation is the bridge from a real-world task to an algorithm. You can now express robotics and AI problems as state-space searches, a skill you will apply in every subsequent search and planning chapter. When you formulate a search problem in later chapters, state the five components explicitly before writing code.

## Key Terms

Problem formulation; initial state; actions; transition model; goal test; path cost; state space; search tree; node; solution; optimal solution; single-state; contingency problem.

## Review Questions

## Multiple-Choice Questions (MCQs)

**q1.** Which of the following is NOT a component of problem formulation?
A. Initial state
B. Heuristic function
C. Goal test
D. Path-cost function

**Answer:** B

**q2.** The state space is best described as:
A. The tree generated during search
B. All states reachable from the initial state
C. The set of all possible actions
D. The optimal solution path

**Answer:** B

## Short-Answer Questions

**1. s1.** Why might a robot-navigation problem be a contingency problem rather than a single-state problem?

## Numerical / Problem-Solving Questions

**1. n1.** A robot moves on a grid. Each step costs 1 energy unit; diagonal steps cost 1.5. A path has 8 orthogonal steps and 2 diagonal steps. What is its total path cost?

## Programming / Design Exercises

**1. p1.** Represent the Romania road network as a Python dictionary mapping each city to its neighbors and distances. Write a function that returns the neighbors and edge cost of any city.

## Mini-Project

State-space poster. For a robotics task of your choice (e.g., robot arm assembly, drone inspection), design the state space: draw 6\u201310 representative states, the actions between them, and identify the goal state(s).

## Further Reading & References

Russell & Norvig (2020), Ch. 3 \u2014 Solving Problems by Searching.

Pearl, J. (1984). Heuristics. Addison-Wesley.

CHAPTER 4

# Uninformed Search Algorithms

*BFS, DFS, UCS, and iterative deepening*

## Chapter Introduction

Once a problem is formulated, we need algorithms to search the state space. Uninformed (blind) search strategies have no knowledge of the goal\u2019s location beyond the problem definition. This chapter presents breadth-first search, depth-first search, uniform-cost search, depth-limited search, and iterative deepening, with complexity analysis, worked traces, and Python implementations. Uninformed strategies are worth mastering because they are the baseline against which every informed heuristic is measured. If you cannot justify that your heuristic search expands fewer nodes than breadth-first search on the same problem, you have not improved anything. The memory behaviour of each strategy matters as much as its speed: a robot controller with kilobytes of RAM cannot run breadth-first search on a large state space, no matter how optimal it is in theory.

## Learning Objectives

## Prerequisite Knowledge

Chapter 3 (problem formulation, state space). Basic Python lists, queues, and recursion.

## Core Concepts

**1. core_a.** General tree search: maintain a frontier (queue of nodes). Repeatedly remove a node; if it is a goal, return the solution; otherwise expand it (apply all actions) and insert the resulting nodes into the frontier. Different strategies differ only in how they order the frontier.

**2. core_b.** Breadth-first search (BFS): expands the shallowest unexpanded node first. Frontier is a FIFO queue. Complete and optimal if step costs are equal; exponential time and space.

**3. core_c.** Depth-first search (DFS): expands the deepest unexpanded node first. Frontier is a LIFO stack. Memory-efficient but neither complete nor optimal in infinite or deep spaces; can get stuck down long branches.

**4. core_d.** Uniform-cost search (UCS): expands the lowest path-cost node first (priority queue ordered by g(n)). Complete and optimal for non-negative costs; can explore many nodes.

**5. core_e.** Depth-limited search (DFS with depth limit) and iterative deepening search (IDS): IDS runs DFS with increasing limits. Combines BFS\u2019s completeness/optimality (unit costs) with DFS-like linear space.

## Key Definitions & Terminology

****Search tree**.**

Tree whose root is the initial state and whose edges are actions.

****Frontier / Open list**.**

Nodes generated but not yet expanded.

****Explored / Closed set**.**

Nodes already expanded.

****Complete**.**

Guaranteed to find a solution if one exists.

****Optimal**.**

Guaranteed to find a least-cost solution.

****Branching factor (b)**.**

Maximum number of successors of any node.

****Depth (d)**.**

Length of the shallowest solution path.

## Worked Examples

**1. example_1.** Worked trace on a tiny 6-node graph (A\u2026F). BFS visits nodes in order A, B, C, D, E, F and finds goal F at shallowest depth; DFS may dive A\u2192B\u2192D then backtrack; UCS accounts for edge costs and may choose a longer-in-edges but cheaper path. Step-by-step frontier contents are shown for each algorithm.

## Algorithms & Pseudocode

- TREE-SEARCH(problem): loop { if frontier empty -> failure; node <- pop(frontier); if problem.GOAL-TEST(node.STATE) -> return node; frontier <- INSERT-ALL(EXPAND(node), problem, frontier) }

- EXPAND(node): for each action, result in problem.ACTIONS(node.STATE): create child node with node.STATE, action, result, node.PATH-COST + step-cost, node.DEPTH + 1.

### Engineering Applications

BFS for finding the fewest-moves solution in a robot puzzle (fewest actions matter).

DFS for exploring a mine shaft where memory is limited but depth is bounded.

UCS for robot path planning where battery/energy cost varies by terrain.

### Practical Laboratory Activity

- Hand-trace BFS and DFS on the Romania map from Arad to Bucharest; record frontier after each expansion.
- Compare nodes expanded by BFS vs. UCS on a weighted grid with uneven terrain costs.

### Programming Exercise

**Listing 4.1: Breadth-first search in Python.**

```
from collections import deque

def bfs(problem):
\"\"\"Breadth-first search. Returns node path or None.\"\"\"
start = Node(problem.initial_state)
if problem.goal_test(start.state): return start
frontier = deque([start])
explored = set()
while frontier:
node = frontier.popleft()
explored.add(node.state)
for action in problem.actions(node.state):
child = Node(problem.result(node.state, action), parent=node,
action=action)
if child.state not in explored and child.state not in [n.state for n in
frontier]:
if problem.goal_test(child.state): return child
frontier.append(child)
return None
```

### Case Study

**1. case_a.** Case: Robot in a burning building must reach an exit. BFS finds the exit through the fewest doors (fewest actions), which may matter more than distance when smoke spreads. UCS would minimize total distance traveled. The choice of search

strategy is an engineering decision tied to the performance measure.

### Common Mistakes & Troubleshooting

- Using DFS on an infinite or very deep state space \u2014 it may never return.
- Using BFS when memory is tight \u2014 frontier grows exponentially.
- Assuming BFS is optimal when step costs differ \u2014 it is not; use UCS.
- Failing to track the explored set, causing repeated states and infinite loops.

### Chapter Summary

Uninformed search gives you a toolbox of blind strategies with clear trade-offs among completeness, optimality, time, and space. You can now implement BFS, DFS, and UCS and choose among them based on the robotics task at hand. Pick the uninformed strategy that matches your memory budget and cost structure; do not default to breadth-first search.

### Key Terms

Tree search; frontier; explored set; breadth-first search; depth-first search; uniform-cost search; depth-limited search; iterative deepening; completeness; optimality; branching factor; FIFO; LIFO; priority queue.

### Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** Which uninformed search is both complete and optimal for unit step costs?
A. DFS
B. BFS
C. Depth-limited search
D. None

**Answer:** B

**q2.** Uniform-cost search orders its frontier by:
A. Depth
B. Path cost g(n)
C. Heuristic h(n)
D. Alphabetical state name

**Answer:** B

**q3.** Iterative deepening search is preferred over BFS when:
A. Memory is limited

B. Step costs are negative
C. The goal is shallow
D. The graph is a tree

**Answer:** A

### Short-Answer Questions

**1. s1.** Why does BFS require exponential memory?

**2. s2.** Can DFS be optimal? Under what conditions?

### Numerical / Problem-Solving Questions

**1. n1.** For a tree with branching factor b = 3 and solution depth d = 4, what is the maximum number of nodes BFS may expand in the worst case?

### Programming / Design Exercises

**1. p1.** Implement DFS (stack-based) and run it on the same Romania problem as BFS; compare the path found and nodes expanded.

### Mini-Project

Maze solver. Implement BFS, DFS, and UCS for a grid maze with movement costs; visualize the explored cells and final path for each algorithm and write a one-page comparison.

### Further Reading & References

Russell & Norvig (2020), Ch. 3 (uninformed search sections).

Sedgewick, R. & Wayne, K. (2011). Algorithms, 4th ed. (Graph search).

CHAPTER 5

# Informed Search Algorithms

*Heuristics, greedy best-first, and A**

## Chapter Introduction

Uninformed search ignores where the goal is. Informed (heuristic) search uses problem-specific knowledge to guide the search toward the goal, often dramatically reducing nodes expanded. This chapter introduces greedy best-first search and A*, heuristic functions, admissibility and consistency, and worked robotics examples. A heuristic is domain knowledge made explicit, and the quality of that knowledge determines the search's usefulness. Manhattan distance is admissible for four-neighbour grid motion because it never overestimates the true path length; doubling it (or using Euclidean distance on a grid where diagonal moves cost the same) would break admissibility and could make A* return a suboptimal route. The worked traces show the frontier contents after every expansion so you can see exactly where the heuristic steers the search.

## Learning Objectives

## Prerequisite Knowledge

Chapters 3\u20134 (state space, uninformed search). Basic geometry (Manhattan and Euclidean distance).

## Core Concepts

**1. core_a.** Informed search: a heuristic h(n) estimates the cost from node n to the nearest goal. It is problem-specific knowledge. A good heuristic is accurate but cheap to compute.

**2. core_b.** Greedy best-first search: expands the node that appears closest to the goal (minimize h(n)). Fast but neither complete nor optimal; can be misled by local obstacles.

**3. core_c.** A* search: evaluates f(n) = g(n) + h(n), where g(n) is cost so far and h(n) is estimated cost to goal. A* is complete and optimal if h is admissible (never overestimates) and, for graph search, consistent (satisfies the triangle inequality). A* is optimally efficient \u2014 no other algorithm using the same heuristic expands fewer nodes.

**4. core_d.** Robotics heuristics: for grid navigation, Manhattan distance (4-neighbor) or Euclidean distance (8-neighbor) to the goal. For robot arms, joint-space distance. These are admissible for unit/terrain costs because they underestimate true cost.

## Key Definitions & Terminology

****Heuristic function h(n)**.**

Estimated cost from node n to the closest goal.

****Admissible**.**

Never overestimates the true cost to the goal.

****Consistent (monotone)**.**

For every node n and successor n\u2019: h(n) \u2264 cost(n,n\u2019) + h(n\u2019).

****Greedy best-first search**.**

Expands node with minimum h(n).

****A* search**.**

Expands node with minimum f(n) = g(n) + h(n).

## Worked Examples

**1. example_1.** On the Romania map, h(n) = straight-line distance to Bucharest. A* expands far fewer nodes than BFS/UCS because it heads toward Bucharest rather than radiating outward. A worked table lists f, g, h for each expanded node.

## Algorithms & Pseudocode

- A*(problem): frontier priority queue ordered by f(n)=g(n)+h(n); pop lowest f; goal test; expand; for each child, if new g is lower, update and reinsert.

## Engineering Applications

A* is the workhorse of robot navigation (used in ROS navigation stack global planner).

GPS routing in autonomous vehicles.

Game AI pathfinding (units moving on maps).

## Practical Laboratory Activity

- Hand-trace A* on a weighted grid with obstacles; compare nodes expanded vs. Dijkstra/UCS.
- Experiment: try an inadmissible heuristic (e.g., 2 x Euclidean distance) and observe that A* returns a suboptimal path.

## Programming Exercise

**Listing 5.1: A* search in Python.**

```
import heapq

def astar(problem, h):
\"\"\"A* search. h is a heuristic function h(state) -> cost
estimate.\"\"\"
start = Node(problem.initial_state, g=0, h=h(problem.initial_state))
frontier = [(start.f, start)] # min-heap by f = g + h
best_g = {start.state: start.g}
while frontier:
_, node = heapq.heappop(frontier)
if problem.goal_test(node.state): return node
for action in problem.actions(node.state):
child = Node(problem.result(node.state, action), parent=node,
g=node.g + problem.step_cost(node.state, action),
h=h(problem.result(node.state, action)))
s = child.state
if s not in best_g or child.g < best_g[s]:
best_g[s] = child.g
heapq.heappush(frontier, (child.f, child))
return None
```

## Case Study

**1. case_a.** Case: ROS navigation global planner. A mobile robot in a warehouse uses A* on an occupancy grid (costmap) with Euclidean distance to the goal as heuristic. It replans when new obstacles appear. Engineers tune the costmap (e.g., higher cost near shelves) so A* routes robots down aisles rather than across them.

## Common Mistakes & Troubleshooting

- Using an inadmissible heuristic and assuming A* still returns the optimal path.
- Computing an expensive heuristic that dominates total runtime \u2014 the heuristic must be cheaper than the search it saves.
- For graph search, forgetting consistency can cause re-expansions; use a closed set or re-open nodes.

## Chapter Summary

Informed search turns blind exploration into directed navigation. You now understand heuristics, admissibility, consistency, greedy best-first, and A*, and can implement A* for robot path planning \u2014 a core skill reused in AI.311 and AI.407. Verify that your heuristic is admissible before trusting A*'s optimality guarantee.

## Key Terms

Heuristic; informed search; greedy best-first search; A*; f(n)=g(n)+h(n); admissible; consistent; optimally efficient; occupancy grid; Manhattan distance; Euclidean distance.

## Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** A* with an admissible heuristic is:
A. Fast but not guaranteed optimal
B. Complete and optimal
C. Neither complete nor optimal
D. Optimal only on trees

**Answer:** B

**q2.** For 4-neighbor grid movement, an admissible heuristic is:
A. Euclidean distance
B. Manhattan distance
C. 2 x Euclidean distance
D. Zero

**Answer:** B

**q3.** Greedy best-first search minimizes:
A. g(n)
B. h(n)
C. f(n)
D. Path length so far

**Answer:** B

### Short-Answer Questions

**1. s1.** Why can greedy best-first search be misled by a local obstacle?

**2. s2.** Name one admissible heuristic for robot-arm joint-space planning.

### Numerical / Problem-Solving Questions

**1. n1.** Node X has g(X)=5 and h(X)=7. Node Y has g(Y)=3 and h(Y)=9. Which has lower f, and which does A* expand next?

### Programming / Design Exercises

**1. p1.** Complete Listing 5.1 with a grid problem (4-neighbor moves, cost 1, Manhattan heuristic) and print the path from start to goal.

## Mini-Project

City navigator. Implement A* on a real (small) street network or occupancy grid; compare path length and nodes expanded against BFS and UCS; submit code plus a one-page report.

## Further Reading & References

Russell & Norvig (2020), Ch. 3 (informed search).

Hart, P., Nilsson, N., & Raphael, B. (1968). \u201cA Formal Basis for the Heuristic Determination of Minimum Cost Paths.\u201d

Stern, R. et al. (2019). \u201cSearch-Based Planning.\u201d Foundations and Trends in Robotics.

CHAPTER 6

# Adversarial Search

*Minimax and alpha-beta pruning*

## Chapter Introduction

So far the environment has been passive or neutral. Adversarial search addresses settings where an opponent acts against us \u2014 games and competitive robotics. This chapter presents minimax and alpha-beta pruning, with worked examples, and discusses how these ideas extend beyond games to negotiation and competitive robot systems. Adversarial search introduces the idea that the environment may act against you, which changes everything about how you evaluate a move. Minimax assumes optimal opposition; real opponents make mistakes, so a perfect minimax player is not always the best practical player. Alpha-beta pruning does not change the answer - it only avoids work - and its savings depend strongly on move ordering, a practical consideration that separates classroom implementations from competitive engines.

## Learning Objectives

## Prerequisite Knowledge

Chapters 3\u20134 (search trees, recursion). Basic game rules (tic-tac-toe) are sufficient.

## Core Concepts

**1. core_a.** Adversarial search: two players, MAX (us) and MIN (opponent), alternate moves in a zero-sum game (one\u2019s gain is the other\u2019s loss). The game is a tree: states are board positions, actions are legal moves, terminal states have utility values (+1 win, 0 draw, -1 loss).

**2. core_b.** Minimax: MAX chooses the move maximizing the minimum utility MIN can force. Values are computed bottom-up from terminal states: MAX nodes take max of children, MIN nodes take min. The result is the optimal move assuming perfect play from both sides.

**3. core_c.** Alpha-beta pruning: an optimization that cuts off branches that cannot influence the root decision. \u03b1 = best value MAX can guarantee; \u03b2 = best value MIN can guarantee. Prune when \u03b1 \u2265 \u03b2. Same result as minimax, fewer nodes evaluated, especially with good move ordering.

**4. core_d.** Depth limits and evaluation functions: real games are too deep to search to termination, so we search to a fixed depth and evaluate non-terminal states with a heuristic evaluation function (e.g., material count in chess).

## Key Definitions & Terminology

****Zero-sum game**.**

A game where total payoff is constant; one player\u2019s gain equals the other\u2019s loss.

****Minimax value**.**

The utility a player can guarantee with optimal play from both sides.

****Alpha (\u03b1)**.**

The value of the best choice found so far for MAX along the path.

****Beta (\u03b2)**.**

The value of the best choice found so far for MIN along the path.

****Evaluation function**.**

A heuristic estimating the utility of a non-terminal state.

## Worked Examples

**1. example_1.** A small 3-ply game tree with terminal values {3, 5, 2, 9, 1, 4}. Compute minimax bottom-up: MAX picks max of MIN children, MIN picks min of MAX children. Then apply alpha-beta to the same tree and show which leaf nodes are never evaluated.

## Algorithms & Pseudocode

- MINIMAX(node): if terminal -> return UTILITY(node); if MAX -> return max(MINIMAX(child)); if MIN -> return min(MINIMAX(child)).
- ALPHA-BETA(node, alpha, beta): propagate alpha/beta; prune child when alpha >= beta.

## Engineering Applications

Game-playing AI (chess, checkers, Go).

Competitive robotics (robot soccer, robot sumo).

Negotiation and auction bidding agents.

## Practical Laboratory Activity

- Play tic-tac-toe against a minimax opponent (implemented or online); observe that perfect play always draws.
- Hand-apply alpha-beta to a provided tree and count pruned leaves vs. full minimax leaf count.

## Programming Exercise

**Listing 6.1: Minimax (depth-limited) in Python.**

```
def minimax(node, depth, maximizing):
if depth == 0 or node.is_terminal():
return node.evaluate()
if maximizing:
best = -inf
for c in node.children():
best = max(best, minimax(c, depth-1, False))
return best
else:
best = +inf
for c in node.children():
best = min(best, minimax(c, depth-1, True))
return best
```

## Case Study

**1. case_a.** Case: Deep Blue / chess engines. Modern engines combine minimax-style search with alpha-beta pruning, deep opening books, endgame tablebases, and neural-network evaluation functions (AlphaZero-style). Engineering lesson: search depth, move ordering, and evaluation quality determine playing strength.

## Common Mistakes & Troubleshooting

- Assuming minimax tells you what the opponent will do (it assumes optimal play; real opponents err).
- Using a poor evaluation function \u2014 search depth cannot compensate for a bad heuristic.
- Forgetting that alpha-beta requires correct move ordering for maximum pruning.

## Chapter Summary

Adversarial search equips you to reason about intelligent opposition. Minimax gives the optimal move under perfect play; alpha-beta makes it feasible. These ideas generalize to any competitive or zero-sum robotics scenario. Remember that minimax assumes optimal opposition and that alpha-beta is an optimisation, not a different answer.

### Key Terms

Adversarial search; zero-sum game; MAX; MIN; minimax; alpha-beta pruning; evaluation function; depth limit; pruning; game tree.

### Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** Minimax assumes the opponent:
A. Plays randomly
B. Plays optimally to minimize our gain
C. Cooperates with us
D. Is irrational

**Answer:** B

**q2.** Alpha-beta pruning:
A. Changes the minimax result
B. Prunes branches that cannot affect the root
C. Only works on trees of depth 2
D. Is slower than minimax

**Answer:** B

### Short-Answer Questions

**1. s1.** Give one non-game application of adversarial search in robotics.

### Numerical / Problem-Solving Questions

**1. n1.** A full binary game tree of depth 3 has how many terminal leaves? (2^3)

### Programming / Design Exercises

**1. p1.** Implement minimax for tic-tac-toe (depth to termination) and let two instances play; confirm all games end in draws.

### Mini-Project

Game AI. Build a minimax-based player for a simple game (nim, tic-tac-toe, or connect-four) with an evaluation function and depth limit; document move-ordering impact on speed.

### Further Reading & References

Russell & Norvig (2020), Ch. 5 \u2014 Adversarial Search.

Newborn, M. (1996). Kasparov vs. Deep Blue. Springer.

CHAPTER 7

# Knowledge Representation and Logic

*Propositional and first-order logic, inference*

## Chapter Introduction

Search finds actions, but many AI tasks require representing what the world is like and drawing conclusions from it. This chapter introduces knowledge representation, propositional logic, and first-order logic, with syntax, semantics, and inference. These underpin expert systems (Chapter 9) and cognitive robotics. Symbolic logic and probabilistic reasoning are complementary, not competing, paradigms. Logic excels when knowledge is certain and discrete (a part is either present or absent); probability excels when it is noisy and graded (a sensor reading is 87 % likely to be correct). Modern cognitive robots use both: logic for high-level task structure, probability for perception and state estimation.

## Learning Objectives

## Prerequisite Knowledge

High-school algebra and basic set notation; truth tables are introduced here.

## Core Concepts

**1. core_a.** Knowledge representation: the study of how to encode facts and rules so a machine can reason with them. A knowledge base (KB) holds sentences; an inference engine derives new sentences. Good representation is expressive, compact, and computable.

**2. core_b.** Propositional logic: propositions are statements that are true or false. Connectives: \u00ac (not), \u2227 (and), \u2228 (or), \u2192 (implies), \u2194 (iff). Truth tables define semantics. Example: R = \u201cit is raining\u201d, W = \u201cthe ground is wet\u201d; R \u2192 W means \u201cif raining then wet.\u201d Limitation: cannot talk about objects or relationships.

**3. core_c.** First-order logic (FOL): adds objects, relations, and quantifiers. Predicates relate objects: Robot(x), At(x, y). Quantifiers: \u2200 (for all), \u2203 (there exists). Example: \u2200 x (Robot(x) \u2192 \u2203 y At(x, y)) = \u201cevery robot is at some location.\u201d FOL is far more expressive and is the basis of many knowledge-based systems.

**4. core_d.** Inference: deriving new sentences from a KB. Modus Ponens: from P and P\u2192Q infer Q. Resolution: a single rule of inference that is complete for propositional logic and can be extended to FOL (with unification).

## Key Definitions & Terminology

****Knowledge base (KB)**.**

A set of sentences representing what the system knows.

****Inference**.**

Deriving new sentences from existing ones.

****Entailment**.**

KB \u226a \u03b1 means \u03b1 is true in all worlds where KB is true.

****Modus Ponens**.**

From P and (P \u2192 Q), conclude Q.

****Unification**.**

Finding substitutions that make two logical expressions identical.

## Worked Examples

**1. example_1.** KB: (1) Robot(Rover). (2) \u2200 x (Robot(x) \u2192 HasBattery(x)). (3) \u2200 x \u2200 y (At(x, y) \u2192 LocatedAt(x, y)). Query: does Rover have a battery? Modus Ponens on (1) and (2) yields HasBattery(Rover).

## Algorithms & Pseudocode

- Forward chaining: start from known facts, apply rules to derive new facts until the query is derived or nothing new results.
- Resolution: convert sentences to conjunctive normal form and derive the empty clause to prove a query by contradiction.

## Engineering Applications

Expert systems (medical diagnosis, equipment fault diagnosis).

Configuration and design assistants.

Semantic reasoning in cognitive robots.

## Practical Laboratory Activity

- Translate ten English sentences into propositional logic and build truth tables for two compound expressions.

- Encode a small robot KB in FOL and answer three queries by forward chaining on paper.

### Programming Exercise

**Listing 7.1: Simple forward chaining (Python sketch).**

```
# Forward chaining on a propositional KB (Python sketch)
kb = {'R': [], 'W': ['R']} # W is true if R is true (R -> W)
facts = {'R'}
agenda = list(facts)
while agenda:
p = agenda.pop()
for q, rules in kb.items():
if p in rules and q not in facts:
facts.add(q); agenda.append(q); print('derived', q)
```

### Case Study

**1. case_a.** Case: Equipment fault-diagnosis expert system. KB contains rules like \u201cif motor-overheated and load-high then reduce-load.\u201d Sensors assert facts; forward chaining derives actions. Used in factory robots to reduce downtime. Limitation: the KB must be authored by experts and is brittle to unforeseen faults.

### Common Mistakes & Troubleshooting

- Treating logic as certain: real robot sensors are noisy, so pure FOL often needs probabilistic extensions (Chapter 8).
- Confusing syntax (form) with semantics (meaning).
- Assuming FOL inference is always efficient; it can be undecidable in general.

### Chapter Summary

You can now represent knowledge in propositional and first-order logic and draw inferences. This symbolic foundation complements the probabilistic and learning methods in later chapters. Choose logic for certain, discrete knowledge and probability for noisy, graded knowledge.

### Key Terms

Knowledge representation; knowledge base; inference; propositional logic; connective; truth table; first-order logic; predicate; quantifier; Modus Ponens; resolution; forward chaining.

### Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** Which connective means \u201cif and only if\u201d?
A. \u00ac
B. \u2227
C. \u2192
D. \u2194

**Answer:** D

**q2.** First-order logic adds to propositional logic:
A. More truth values
B. Objects, relations, and quantifiers
C. Only negation
D. Probability

**Answer:** B

### Short-Answer Questions

**1. s1.** Why can\u2019t propositional logic express \u201call robots have batteries\u201d compactly?

### Numerical / Problem-Solving Questions

**1. n1.** A truth table for 3 propositions has how many rows?

### Programming / Design Exercises

**1. p1.** Extend Listing 7.1 to support rules with multiple premises (AND of facts).

### Mini-Project

Knowledge base builder. Encode a small domain (e.g., robot course prerequisites or a household device troubleshooting guide) as logical rules and implement a forward-chaining query tool.

### Further Reading & References

Russell & Norvig (2020), Ch. 7\u20138 \u2014 Logical Agents, First-Order Logic.

Brachman, R. & Levesque, H. (2004). Knowledge Representation and Reasoning. Morgan Kaufmann.

CHAPTER 8

# Reasoning Under Uncertainty and Bayesian AI

*Probability, Bayes' theorem, decision-making*

## Chapter Introduction

Real robots operate with noisy sensors and uncertain effects. Pure logic cannot say \u201cprobably\u201d or \u201chow likely.\u201d This chapter introduces probability as the language of uncertainty, Bayesian reasoning, and decision-making under uncertainty \u2014 essential for sensor fusion, perception, and autonomous robots. Bayes' theorem is the single most important equation in this book for robotics. It formalises the intuition that evidence should update belief in proportion to how expected that evidence was under each hypothesis. The worked fault-diagnosis example shows a surprising and important result: even a highly accurate alarm is usually wrong when the underlying fault is rare. Alarm design, sensor fusion, and diagnostic reasoning all depend on grasping this base-rate effect.

## Learning Objectives

## Prerequisite Knowledge

Basic probability (coin flips, dice) is helpful; all formulas are derived and explained.

## Core Concepts

**1. core_a.** Probability basics: P(A) is the probability of event A. Conditional probability P(A|B) = P(A \u2229 B)/P(B). Joint distribution P(A,B). Marginalization: P(A) = \u2211_b P(A, b).

**2. core_b.** Bayes\u2019 theorem: P(A|B) = P(B|A)P(A) / P(B). It lets us update a prior belief P(A) with evidence B to get a posterior P(A|B). This is the core of probabilistic perception and filtering (revisited in AI.310).

**3. core_c.** Random variables and distributions: discrete and continuous; expectation; variance. Example: a robot\u2019s location is a random variable over grid cells.

**4. core_d.** Decision-making under uncertainty: choose the action maximizing expected utility EU(a) = \u2211_s P(s|a) U(s). Combines beliefs (probabilities) with preferences (utilities).

## Key Definitions & Terminology

****Prior probability**.**

P(A) before seeing evidence.

****Posterior probability**.**

P(A|B) after observing evidence B.

****Likelihood**.**

P(B|A): probability of evidence given the hypothesis.

****Expected utility**.**

The utility-weighted average outcome of an action.

## Worked Examples

**1. example_1.** Medical-test style robot example: a fault sensor has P(alarm|fault)=0.9, P(alarm|no fault)=0.1, and P(fault)=0.02. Given an alarm, Bayes gives P(fault|alarm) = (0.9\u00d70.02)/[(0.9\u00d70.02)+(0.1\u00d70.98)] \u2248 0.155. Even with an alarm, the fault is only ~15.5% likely because faults are rare \u2014 a crucial engineering insight for alarm design.

## Algorithms & Pseudocode

- Bayesian update: posterior \u2248 likelihood \u00d7 prior / evidence.
- Maximum a posteriori (MAP): choose hypothesis maximizing posterior.

## Engineering Applications

Sensor fusion: combine camera and LiDAR estimates of object position.

Spam filtering (na\u00efve Bayes, Chapter 10).

Medical diagnosis and fault diagnosis.

## Practical Laboratory Activity

- Compute posterior fault probability for varying prior fault rates (1%, 5%, 10%) and observe sensitivity.
- Simulate a noisy binary sensor and estimate P(fault|alarm) by Monte Carlo counting.

## Programming Exercise

**`Listing 8.1: Bayesian fault diagnosis (Python).`**

```
# Bayes update for robot fault diagnosis
p_fault = 0.02
p_alarm_given_fault = 0.9
p_alarm_given_no_fault = 0.1
p_alarm = (p_alarm_given_fault * p_fault +
p_alarm_given_no_fault * (1 - p_fault))
p_fault_given_alarm = (p_alarm_given_fault * p_fault) / p_alarm
print('P(fault|alarm) =', round(p_fault_given_alarm, 4))
```

## Case Study

**1. case_a.** Case: Autonomous car at an intersection. A camera detects a pedestrian with some uncertainty; radar gives a different estimate. Bayesian fusion combines both into a posterior over \u201cpedestrian present,\u201d which feeds the braking decision. Expected utility weighs braking (safe but uncomfortable) against not braking (risky).

## Common Mistakes & Troubleshooting

- Confusing P(A|B) with P(B|A) \u2014 the prosecutor\u2019s fallacy. Bayes\u2019 theorem is the remedy.
- Ignoring the prior: a rare fault makes even a strong alarm unreliable.
- Treating sensor readings as certain when they are noisy.

## Chapter Summary

Probability and Bayesian reasoning give AI a rigorous way to handle uncertainty. You can now update beliefs with evidence and make decisions by expected utility \u2014 tools used throughout perception, planning, and robotics. Always report the prior when interpreting an alarm or a diagnostic result.

## Key Terms

Probability; conditional probability; joint distribution; Bayes\u2019 theorem; prior; posterior; likelihood; expected utility; decision under uncertainty; sensor fusion.

## Review Questions

## Multiple-Choice Questions (MCQs)

**q1.** Bayes\u2019 theorem computes:
A. The prior
B. The posterior from prior, likelihood, and evidence
C. Only joint probabilities

D. Expected utility

**Answer:** B

**q2.** Expected utility of an action is:
A. Its most likely outcome
B. The probability-weighted average of utilities
C. Always positive
D. Independent of beliefs

**Answer:** B

### Short-Answer Questions

**1. s1.** Why might an autonomous car still brake after a low-probability pedestrian detection?

### Numerical / Problem-Solving Questions

**1. n1.** Recompute P(fault|alarm) if P(fault) rises to 0.10 (sensor characteristics unchanged).

### Programming / Design Exercises

**1. p1.** Generalize Listing 8.1 to accept any prior, likelihoods, and observed evidence, and print the posterior.

### Mini-Project

Uncertainty dashboard. Implement a Bayesian updater for a robot sensor (e.g., door open/closed) and plot posterior vs. number of observations.

### Further Reading & References

Russell & Norvig (2020), Ch. 13\u201314 \u2014 Uncertainty, Probabilistic Reasoning.

Pearl, J. (1988). Probabilistic Reasoning in Intelligent Systems. Morgan Kaufmann.

CHAPTER 9

# Expert Systems

*Rule-based diagnosis and the match-resolve-act cycle*

## Chapter Introduction

Expert systems were among the first successful commercial AI applications. They encode human expertise as rules and use an inference engine to reach conclusions. This chapter covers rule-based systems, forward/backward chaining, knowledge acquisition, and includes the rule-based expert-system laboratory required by the course prompt. Expert systems illustrate both the power and the brittleness of rule-based AI. They are transparent and explainable - you can always ask which rule fired and why - but they fail on anything outside their rules, and acquiring those rules from human experts is slow and error-prone. Modern systems usually combine rules (for structure and explainability) with learning (for adaptation) and probability (for uncertainty), exactly the hybrid this textbook builds toward.

## Learning Objectives

## Prerequisite Knowledge

Chapter 7 (logic, forward chaining).

## Core Concepts

**1. core_a.** Architecture: knowledge base (facts + rules), inference engine, working memory (current facts), user interface, and explanation facility (why/how questions).

**2. core_b.** Production rules: IF condition THEN action. Conditions match facts in working memory; actions assert new facts or fire actions.

**3. core_c.** Forward chaining (data-driven): assert facts, fire matching rules, repeat until goal or quiescence. Good for diagnosis with many possible conclusions.

**4. core_d.** Backward chaining (goal-driven): start from the goal, find rules that conclude it, recursively prove their premises. Good when few goals are of interest.

**5. core_e.** Conflict resolution: when multiple rules fire, choose one (e.g., most specific, most recently used, highest priority).

## Key Definitions & Terminology

****Production rule**.**

IF THEN .

****Working memory**.**

The set of currently asserted facts.

****Inference engine**.**

The mechanism that selects and fires rules.

****Conflict resolution**.**

Choosing among multiple fireable rules.

****Knowledge acquisition**.**

Extracting and encoding expert knowledge.

## Worked Examples

**1. example_1.** A small robot troubleshooting KB: IF motor-hot AND load-high THEN reduce-load; IF reduce-load AND motor-still-hot THEN shutdown; IF obstruction-detected THEN clear-obstruction. Given facts {motor-hot, load-high}, forward chaining derives reduce-load then shutdown.

## Algorithms & Pseudocode

- Match\u2013Resolve\u2013Act cycle: match rules to working memory, resolve conflicts, act (assert/retract), repeat.

## Engineering Applications

Medical diagnosis (MYCIN-style), equipment fault diagnosis, configuration, credit approval.

Robot fault diagnosis and predictive maintenance hints (Chapter 20).

## Practical Laboratory Activity

- Practical (required): build a rule-based expert system in Python for a chosen domain (e.g., robot fault diagnosis or course-advising) with at least 8 rules; demonstrate forward chaining on two scenarios; include an explanation trace (which rules fired).

## Programming Exercise

**Listing 9.1: Rule-based expert system (forward chaining, Python).**

```
# Minimal rule-based expert system (forward chaining)
rules = [
```

```
({"motor-hot", "load-high"}, "reduce-load"),
({"reduce-load", "motor-hot"}, "shutdown"),
({"obstruction"}, "clear-obstruction"),
]
wm = {"motor-hot", "load-high"} # observed facts
fired = []
changed = True
while changed:
changed = False
for cond, act in rules:
if cond <= wm and act not in wm:
wm.add(act); fired.append((cond, act)); changed = True
print('Facts:', sorted(wm))
print('Rules fired:', fired)
```

## Case Study

**1. case_a.** Case: Industrial robot fault diagnosis. A robot cell reports sensor faults; the expert system maps symptom combinations to likely causes (encoder failure, cable wear, controller fault) and recommended actions. Engineers maintain the KB; the explanation facility tells operators why a cause was selected.

## Common Mistakes & Troubleshooting

- Rule explosion: too many rules become hard to maintain and conflict-prone.
- Brittleness: the system fails on situations outside its rules.
- Knowledge-acquisition bottleneck: experts struggle to articulate all their rules.

## Chapter Summary

Expert systems show how encoded expertise plus inference can automate diagnosis. You have built one. Modern systems augment rules with learning (Chapter 10) and probability (Chapter 8), but the rule-based pattern remains widely used. Treat rule-based systems as explainable but brittle, and plan for the hybrid systems that follow.

## Key Terms

Expert system; knowledge base; inference engine; working memory; production rule; forward chaining; backward chaining; conflict resolution; knowledge acquisition; match-resolve-act.

## Review Questions

### Multiple-Choice Questions (MCQs)

**q1.** Forward chaining is:
A. Goal-driven
B. Data-driven
C. Only for classification
D. The same as backward chaining

**Answer:** B

**q2.** The match-resolve-act cycle is part of:
A. The knowledge base
B. The inference engine
C. The user interface
D. Knowledge acquisition

**Answer:** B

### Short-Answer Questions

**1. s1.** Name one domain where expert systems are still used today.

### Numerical / Problem-Solving Questions

**1. n1.** If 6 of 12 rules match working memory and conflict resolution picks the highest-priority one, how many rules fire in that cycle?

### Programming / Design Exercises

**1. p1.** Add backward chaining to a small KB and answer a goal query.

### Mini-Project

Diagnostic expert system. Build a rule-based system that diagnoses a robotics or AI fault from symptom inputs, with explanation output; test on five cases and report accuracy.

### Further Reading & References

Russell & Norvig (2020), Ch. 9 (expert systems material).

Giarratano, J. & Riley, G. (2004). Expert Systems: Principles and Programming. Cengage.

Buchanan, B. & Shortliffe, E. (eds.) (1984). Rule-Based Expert Systems. Addison-Wesley.

CHAPTER 10

# Machine Learning and Neural Networks Overview

*Supervised learning, classification, overfitting*

## Chapter Introduction

Machine learning lets systems improve from data rather than explicit rules. This chapter gives a supervised-learning and neural-network overview \u2014 classification, regression, training/test split, a basic classifier lab, and a glimpse of neural networks \u2014 preparing for the dedicated AI.103 and AI.111 courses. Machine learning shifts the engineer's job from writing rules to designing data, features, and evaluation. A model that scores 100 % on its training data and 60 % on new data has memorised, not learned. The train/validation/test split is not bureaucracy: it is the safeguard that lets you report performance you can actually trust. This overview is intentionally shallow; AI.103 and AI.111 cover learning and networks in depth.

## Learning Objectives

## Prerequisite Knowledge

Chapter 8 (probability). Basic Python; NumPy introduced as needed.

## Core Concepts

**1. core_a.** Supervised learning: learn a mapping from inputs (features) to outputs (labels) from labeled examples. Classification (discrete labels) and regression (continuous targets). Example: classify email as spam/not-spam; predict house price from size.

**2. core_b.** Unsupervised learning: find structure in unlabeled data (clustering, dimensionality reduction). Reinforcement learning: learn from reward/penalty (deeper in AI.205).

**3. core_c.** Training, validation, test sets: train on training data, tune hyperparameters on validation, report final performance on held-out test data. Never tune on the test set.

**4. core_d.** Overfitting: model memorizes training noise and fails to generalize. Remedies: more data, simpler model, regularization, cross-validation.

**5. core_e.** Neural network overview: neurons compute weighted sum + activation; layers form a network; backpropagation learns weights (detailed in AI.111). A basic classifier lab uses scikit-learn.

## Key Definitions & Terminology

****Feature**.**

A measurable input property (e.g., email word count).

****Label**.**

The target output (e.g., spam / not-spam).

****Training set**.**

Labeled data used to fit the model.

****Test set**.**

Held-out data used to estimate generalization.

****Overfitting**.**

Good training performance, poor test performance.

## Worked Examples

**1. example_1.** Spam classification: features = presence of words (\u201cfree\u201d, \u201cmoney\u201d, \u201curgent\u201d); label = spam/not-spam. Train a Na\u00efve Bayes or logistic classifier on labeled emails; test on new emails. This is the \u201cbasic classifier\u201d lab.

## Algorithms & Pseudocode

- k-Nearest Neighbors: classify by majority vote of the k closest training examples.
- Perceptron: linear classifier that updates weights on mistakes.

## Engineering Applications

Robotic sensor classification (Chapter 10 lab): classify terrain or object type from sensor readings.

Spam filtering, image recognition, recommendation.

## Practical Laboratory Activity

- Practical (required): develop a basic classifier (scikit-learn) on a dataset such as Iris or a robot-sensor dataset; report train/test accuracy and a confusion matrix.

## Programming Exercise

**Listing 10.1: Basic classifier with scikit-learn.**

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
clf = GaussianNB().fit(X_train, y_train)
print('Test accuracy:', accuracy_score(y_test, clf.predict(X_test)))
```

## Case Study

**1. case_a.** Case: Robotic sensor classification. A robot\u2019s IR/ultrasonic readings are labeled by terrain (carpet/tile/concrete). A classifier trained on labeled readings predicts terrain onboard, helping the robot choose speed. Shows perception feeding action selection.

## Common Mistakes & Troubleshooting

- Training and testing on the same data (overoptimistic accuracy).
- Ignoring class imbalance (accuracy can be high while missing the minority class).
- Using raw, unnormalized features for distance-based classifiers.

## Chapter Summary

You have seen supervised learning end-to-end and trained a basic classifier. This overview is expanded in AI.103 (Machine Learning) and AI.111 (Neural Networks). Never report training-set accuracy as the model's performance; always use a held-out test set.

## Key Terms

Machine learning; supervised learning; classification; regression; training set; test set; overfitting; feature; label; Na\u00efve Bayes; neural network; scikit-learn.

## Review Questions

## Multiple-Choice Questions (MCQs)

**q1.** Supervised learning requires:
A. Labeled data
B. Unlabeled data
C. Rewards only

D. No data

**Answer:** A

**q2.** Overfitting means:
A. Poor training and test performance
B. Good training, poor test performance
C. Too little training data
D. Too many features

**Answer:** B

## Short-Answer Questions

**1. s1.** Name one unsupervised learning task.

## Numerical / Problem-Solving Questions

**1. n1.** A classifier is correct on 42 of 50 test emails. What is test accuracy?

## Programming / Design Exercises

**1. p1.** Train a k-NN classifier (scikit-learn) on Iris and vary k; plot accuracy vs. k.

## Mini-Project

Classifier comparison. Train three classifiers (Na\u00efve Bayes, k-NN, logistic regression) on the same dataset and compare test accuracy and confusion matrices in a short report.

## Further Reading & References

Murphy, K. P. (2022). Probabilistic Machine Learning. MIT Press.

Hastie, T. et al. (2009). The Elements of Statistical Learning. Springer.

CHAPTER 11

# Natural Language Processing and Computer Vision

*Chatbots, pixels, edges, and image understanding*

## Chapter Introduction

Two of AI\u2019s most visible capabilities are understanding language and understanding images. This chapter surveys natural language processing (tokenization, chatbots) and computer vision (images, edges, classification), including the simple-chatbot laboratory and a vision-oriented application, preparing for AI.104 and AI.105. Language and vision are the two AI capabilities most visible to end users, and both are pipelines of deterministic stages followed by a learned or rule-based decision. The chatbot laboratory is required precisely because building one exposes every limitation of pattern matching: no generalisation, no memory beyond the current reply, and brittle failure on unfamiliar input. Treat it as a baseline, not as conversational AI.

## Learning Objectives

## Prerequisite Knowledge

Chapters 1\u20132. Basic Python strings and lists; images introduced as arrays.

## Core Concepts

**1. core_a.** NLP pipeline: text \u2192 tokenization (split into words/sentences) \u2192 cleaning (lowercase, stop-word removal) \u2192 representation (bag-of-words/TF-IDF) \u2192 model (classifier or chatbot rules). A simple chatbot matches user input to patterns and returns templated replies.

**2. core_b.** Computer vision: images are grids of pixels; color spaces (RGB, grayscale, HSV); convolution and filtering; edge detection (gradients); image classification via features or neural networks (CNNs introduced in AI.210).

**3. core_c.** Vision in robotics: a robot camera provides images; vision extracts objects, lines, or faces that guide navigation or manipulation.

## Key Definitions & Terminology

****Tokenization**.**

Splitting text into words or sentences.

****Bag-of-words**.**

Text represented as word counts, ignoring order.

****Pixel**.**

A single picture element with color/intensity values.

****Convolution**.**

A filtering operation using a small kernel over an image.

****Edge detection**.**

Finding boundaries where intensity changes sharply.

## Worked Examples

**1. example_1.** A rule-based chatbot: user says \u201chello\u201d \u2192 reply \u201cHello! How can I help?\u201d; user says \u201cbye\u201d \u2192 \u201cGoodbye!\u201d; otherwise \u2192 \u201cI don\u2019t understand.\u201d Demonstrates pattern matching and response selection.

## Algorithms & Pseudocode

- Chatbot: normalize input \u2192 match against pattern list \u2192 return matched reply or default.
- Edge detection: compute horizontal/vertical gradients (e.g., Sobel) and threshold magnitude.

## Engineering Applications

Voice/text interface for robots (NLP for robots, AI.208).

Object detection and face detection for robot perception (AI.104, AI.406).

## Practical Laboratory Activity

- Practical (required): create a simple chatbot in Python (rule-based or using a tiny intent classifier) that holds a short dialogue.
- Optional vision: load an image, convert to grayscale, apply edge detection, and display edges (OpenCV introduced in AI.104).

## Programming Exercise

**Listing 11.1: Tiny rule-based chatbot (Python).**

```
# Tiny rule-based chatbot
patterns = {
"hello": "Hello! How can I help you?",
"bye": "Goodbye! Have a great day.",
"name": "I am a simple chatbot.",
}
def reply(text):
t = text.lower()
for key, ans in patterns.items():
if key in t: return ans
return "I don\u2019t understand yet."

print(reply("Hello there"))
```

## Case Study

**1. case_a.** Case: Service-robot voice interface. A receptionist robot transcribes speech (speech recognition), parses intent (NLP), and replies (text-to-speech). Errors are handled with clarification requests. Shows NLP embedded in HRI (AI.110).

## Common Mistakes & Troubleshooting

- Expecting a rule-based chatbot to handle arbitrary conversation (it cannot generalize).
- Treating an image as a single number \u2014 it is a 2D/3D array.
- Ignoring lighting and viewpoint in vision applications.

## Chapter Summary

NLP and computer vision give robots language and sight. You have built a chatbot and seen the vision pipeline. These are developed fully in AI.104, AI.105, and AI.406. View the chatbot and vision pipeline as baselines whose limitations are as important as their capabilities.

## Key Terms

NLP; tokenization; bag-of-words; chatbot; pixel; RGB; grayscale; convolution; edge detection; computer vision; speech recognition; text-to-speech.

## Review Questions

## Multiple-Choice Questions (MCQs)

**q1.** Tokenization is:
A. Removing stop words

B. Splitting text into tokens
C. Classifying sentiment
D. Converting speech to text

**Answer:** B

**q2.** A grayscale image pixel has:
A. Three values (RGB)
B. One intensity value
C. No values
D. Four values

**Answer:** B

### Short-Answer Questions

**1. s1.** Give one way computer vision helps a mobile robot.

### Numerical / Problem-Solving Questions

**1. n1.** A 640x480 color image has how many pixels?

### Programming / Design Exercises

**1. p1.** Extend the chatbot with two new intents and test the dialogue.

### Mini-Project

Dialogue + vision demo. Combine the chatbot with an image input (e.g., classify a shown object and reply) in a simple script; document what works and what fails.

### Further Reading & References

Jurafsky, D. & Martin, J. H. (2023). Speech and Language Processing. (draft 3rd ed.)

Forsyth, D. & Ponce, J. (2022). Computer Vision: A Modern Approach. Pearson.

CHAPTER 12

# Robotics, Generative AI, Responsible AI, and the Future

*Integration, emerging tech, ethics, and final project*

## Chapter Introduction

This concluding chapter connects AI to robotics and autonomous systems, surveys generative AI and emerging AI technologies, reviews AI applications and limitations, and addresses responsible AI \u2014 safety, bias, privacy, and societal implications \u2014 required by the course prompt. It ends with the final integrated AI project guidance. This chapter ties the textbook together: perception, reasoning, learning, and action are not separate subjects but stages of one system. Generative AI, foundation models, multimodal AI, and edge AI are described honestly as emerging capabilities with real limitations, not as solved problems. Responsible AI - safety, bias, privacy, explainability, accountability - is treated as a core engineering constraint, not an afterthought, because autonomous systems operate in the physical world where failures can injure people.

## Learning Objectives

## Prerequisite Knowledge

All preceding chapters (1\u201311).

## Core Concepts

**1. core_a.** AI in robotics: perception (vision, LiDAR), reasoning (planning, search), learning (classification, RL), and action (control, manipulation) combine in autonomous robots (AI.208, AI.403).

**2. core_b.** Generative AI: models that create new content (text, images, code) from learned distributions; large language models and diffusion models are leading examples. Emerging AI: foundation models, multimodal AI, edge AI/TinyML, neuromorphic computing.

**3. core_c.** AI applications: manufacturing, healthcare, agriculture, autonomous vehicles, service robots, finance. Limitations: narrowness, brittleness, data hunger, lack of common sense, opacity, safety and fairness risks.

**4. core_d.** Responsible AI: safety (robust, predictable behavior), bias (fair outcomes across groups), privacy (data protection), transparency/explainability, accountability, and societal impact (employment, accessibility, environment).

## Key Definitions & Terminology

****Generative AI**.**

AI that generates new content (text, image, audio) from learned distributions.

****Foundation model**.**

A pre-trained model adaptable to many tasks.

****Bias**.**

Systematic unfairness in system outcomes across groups.

****Explainability**.**

The degree to which a system\u2019s decisions can be understood.

****Autonomous system**.**

A system that senses, decides, and acts with limited human supervision.

## Worked Examples

**1. example_1.** Generative AI in robotics: an LLM translates a user\u2019s spoken instruction (\u201cbring me the red cup\u201d) into a sequence of robot actions (navigate to kitchen, detect cup, grasp, return). Shows language grounding to action.

## Algorithms & Pseudocode

- No new algorithm; this chapter synthesizes prior search, logic, probability, and learning concepts into system design.

## Engineering Applications

Autonomous vehicles, warehouse robots, surgical assistants, agricultural robots, surveillance.

## Practical Laboratory Activity

- Practical: build a simple intelligent agent that senses a simulated environment, decides with a rule base or small classifier, and acts \u2014 tying together perception, decision, and action.
- Responsible-AI exercise: audit an AI system for bias using a small dataset (e.g., compare error rates across groups).

## Programming Exercise

**Listing 12.1: Simple intelligent agent (Python).**

```
# Simple intelligent agent: sense -> decide -> act
def agent(percept):
# percept: dict with sensor readings
if percept.get('obstacle_ahead'):
return 'stop'
if percept.get('battery_low'):
return 'return_to_base'
return 'explore'

for p in [{'obstacle_ahead': True}, {'battery_low': True}, {}]:
print(p, '->', agent(p))
```

## Case Study

**1. case_a.** Case: AI-powered agricultural robot. It perceives crops (vision), decides weed vs. crop (classifier), acts (mechanical weeding), and must operate safely near farmworkers (responsible AI/safety). Illustrates full AI\u2014robotics integration and real-world constraints.

## Common Mistakes & Troubleshooting

- Deploying AI without testing edge cases (safety).
- Ignoring bias in training data (unfair outcomes).
- Collecting personal data without consent (privacy).

## Chapter Summary

You have completed a survey of AI: foundations, agents, search, logic, probability, expert systems, learning, NLP, vision, and robotics. You are prepared for the specialized Year 1\u20134 courses and for the final integrated project below. Treat responsible AI as a design constraint from the start of any autonomous-system project, not a compliance step at the end.

## Key Terms

Robotics; autonomous system; generative AI; foundation model; multimodal AI; edge AI; TinyML; bias; privacy; explainability; accountability; responsible AI; societal impact.

## Review Questions

## Multiple-Choice Questions (MCQs)

**q1.** Generative AI is best described as:
A. Classifying images
B. Generating new content from learned distributions
C. Searching state spaces
D. Controlling robot arms

**Answer:** B

**q2.** Which is NOT a responsible-AI concern?
A. Safety
B. Bias
C. Ignoring user privacy
D. Explainability

**Answer:** C

### Short-Answer Questions

**1. s1.** Why is explainability important when an AI system makes high-stakes decisions?

### Numerical / Problem-Solving Questions

**1. n1.** An agricultural robot classifies 200 plants; 180 are correct. What is accuracy?

### Programming / Design Exercises

**1. p1.** Extend Listing 12.1 with more sensor conditions and actions; simulate a short run.

### Mini-Project

Responsible-AI audit. In groups, audit one AI system (from this book or a real product) for safety, bias, privacy, and societal impact; produce a short audit report with recommendations.

### Further Reading & References

Russell & Norvig (2020), Ch. 27\u201328 (philosophy, ethics, future of AI).

Jobin, A. et al. (2019). \u201cThe Global Landscape of AI Ethics Guidelines.\u201d Nature Machine Intelligence.

Floridi, L. & Cowls, J. (2019). \u201cA Unified Framework of Five Principles for AI in Society.\u201d

PART IV

# Final Integrated Course Project

*Final Integrated AI Project: Autonomous Delivery Agent*

Design, implement, and document a small autonomous intelligent agent that navigates a simulated grid environment, perceives obstacles and a delivery target, plans a path (A* or BFS), avoids obstacles, and reports status. Deliverables: code, a short report, a demonstration, and a reflection on responsible AI (safety, bias, privacy).

### Project Phases

- Phase 1 \u2014 Problem formulation: define states, actions, goal, cost (Chapter 3).
- Phase 2 \u2014 Search: implement BFS and A* for path planning (Chapters 4\u20135).
- Phase 3 \u2014 Agent: wrap planning in a sense-decide-act loop (Chapter 2/12).
- Phase 4 \u2014 Uncertainty: add noisy sensor handling with Bayesian update (Chapter 8).
- Phase 5 \u2014 Documentation & ethics: report, demo, responsible-AI reflection (Chapter 12).

### Assessment Rubric

- Correct problem formulation (15%).
- Working search implementation with tests (25%).
- Agent loop and obstacle handling (20%).
- Code quality and documentation (15%).
- Report and demonstration (15%).
- Responsible-AI reflection (10%).

# Back Matter

## Glossary

| Term | Definition |
|---|---|
| **Agent** | An entity that perceives its environment through sensors and acts through actuators. |
| **Artificial Intelligence (AI)** | The study and construction of systems that perform tasks requiring intelligence. |
| **A* search** | Informed search minimizing f(n) = g(n) + h(n); complete and optimal with an admissible heuristic. |
| **Alpha-beta pruning** | Minimax optimization that skips branches that cannot affect the root decision. |
| **Bayes\u2019 theorem** | P(A\|B) = P(B\|A)P(A)/P(B); updates prior beliefs with evidence. |
| **Breadth-first search (BFS)** | Uninformed search expanding the shallowest node first; complete and optimal for unit costs. |
| **Chatbot** | A conversational agent, rule-based or learned. |
| **Computer vision** | Computational extraction of information from images and video. |
| **Depth-first search (DFS)** | Uninformed search expanding the deepest node first; memory-efficient but not complete/optimal in general. |
| **Expert system** | A knowledge-based system that reasons with encoded expert rules. |
| **Forward chaining** | Data-driven inference: fire rules on known facts to derive new facts. |
| **Generative AI** | AI that creates new content from learned distributions. |
| **Heuristic** | An estimate used to guide search toward a goal. |
| **Intelligent agent** | An agent that acts to achieve goals, possibly learning and adapting. |

| Term | Definition |
|---|---|
| **Machine learning** | AI systems that improve through experience with data. |
| **Minimax** | Optimal decision rule for two-player zero-sum games. |
| **Modus Ponens** | From P and P\u2192Q, infer Q. |
| **Natural language processing (NLP)** | Computational processing of human language. |
| **Overfitting** | Model that fits training data but fails to generalize. |
| **Rational agent** | An agent that acts to maximize expected performance given its knowledge. |
| **Responsible AI** | AI development attentive to safety, bias, privacy, transparency, and societal impact. |
| **State space** | All states reachable from a problem\u2019s initial state. |
| **Uniform-cost search (UCS)** | Uninformed search expanding the lowest path-cost node first. |

## Bibliography

**[1]** Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th ed. Pearson.

**[2]** Nilsson, N. J. (2010). The Quest for Artificial Intelligence. Cambridge University Press.

**[3]** Turing, A. M. (1950). \u201cComputing Machinery and Intelligence.\u201d Mind, 59(236), 433\u2013460.

**[4]** McCarthy, J. et al. (1955). A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.

**[5]** Pearl, J. (1984). Heuristics: Intelligent Search Strategies for Computer Problem Solving. Addison-Wesley.

**[6]** Pearl, J. (1988). Probabilistic Reasoning in Intelligent Systems. Morgan Kaufmann.

**[7]** Murphy, K. P. (2022). Probabilistic Machine Learning: An Introduction. MIT Press.

**[8]** Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning, 2nd ed. Springer.

**[9]** Jurafsky, D. & Martin, J. H. (2023). Speech and Language Processing, 3rd ed. draft.

**[10]** Forsyth, D. & Ponce, J. (2022). Computer Vision: A Modern Approach, 2nd ed. Pearson.

**[11]** Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. Nature Machine Intelligence, 1, 389\u2013399.

**[12]** Floridi, L. & Cowls, J. (2019). A unified framework of five principles for AI in society. Harvard Data Science Review.

# Subject Index

**A***… 45\u201352

**Adversarial search**… 53\u201360

**Alpha-beta pruning**… 56\u201358

**Agents**… 12\u201320

**Bayes\u2019 theorem**… 75\u201380

**Breadth-first search**… 31\u201336

**Computer vision**… 96\u2013102

**Depth-first search**… 36\u201340

**Expert systems**… 69\u201376

**First-order logic**… 63\u201368

**Generative AI**… 113\u2013116

**Heuristics**… 46\u201352

**Machine learning**… 89\u201395

**Minimax**… 54\u201356

**Natural language processing**… 96\u2013102

**Neural networks**… 89\u201395

**Responsible AI**… 113\u2013116

**Search**… 27\u201360

**State space**… 27\u201330

**Uniform-cost search**… 40\u201344

# Practical-Laboratory Index

| Laboratory Activity | Chapter |
|---|---|
| Lab 1.1 \u2014 Reactive robot observation | Chapter 1 |
| Lab 2.1 \u2014 PEAS tables and simple-reflex vacuum | Chapter 2 |
| Lab 3.1 \u2014 State-space: 8-puzzle and warehouse robot | Chapter 3 |
| Lab 4.1 \u2014 BFS/DFS/UCS traces and maze solving | Chapter 4 |
| Lab 5.1 \u2014 A* grid navigation and inadmissible heuristic | Chapter 5 |
| Lab 6.1 \u2014 Minimax tic-tac-toe | Chapter 6 |
| Lab 7.1 \u2014 Forward-chaining knowledge base | Chapter 7 |
| Lab 8.1 \u2014 Bayesian fault diagnosis | Chapter 8 |
| Lab 9.1 \u2014 Rule-based expert system | Chapter 9 |
| Lab 10.1 \u2014 Basic classifier (scikit-learn) | Chapter 10 |
| Lab 11.1 \u2014 Chatbot and (optional) edge detection | Chapter 11 |
| Lab 12.1 \u2014 Simple intelligent agent and responsible-AI audit | Chapter 12 |
| Final integrated project \u2014 Autonomous delivery agent | Final Project |

## Recommended Software and Hardware Tools

| Category | Tools |
|---|---|
| **Programming** | Python 3.11+; NumPy; scikit-learn |
| **Version control** | Git |
| **Editor/IDE** | VS Code, PyCharm, or Jupyter |
| **Robotics simulation (later courses)** | ROS/ROS 2, Gazebo, Webots, PyBullet |
| **Computer vision (later courses)** | OpenCV |
| **Documentation** | Markdown, LaTeX, or Word for reports |

*End of textbook — TRIUNE BIBLICAL UNIVERSITY GLOBAL EXTENSION USA © 2026*